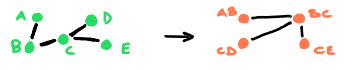


Graph definition $\rightarrow$ a graph can be defined using a pair (V, E) where V is the set of vertices and E is the set of edges
if we take $E = \{(u, v) \in V^2 \mid (u, v) \in E \wedge (v, u) \in E\}$ we have an undirected graph otherwise we have a directed graph
We define a **simple graph** as an undirected graph with no self loops
Two nodes are called **adjacent** if they have exactly one common neighbor
for example $\rightarrow$  a and c are adjacent
Order and Size of a graph $\rightarrow$ the order is the number of vertices $|V|$ and the size is the number of edges $|E|$, for example  has order = 4 and size = 5.
Density of a graph $\rightarrow$ it is the ratio between the graph's size and the number of all possible nodes, we use two different formulas for directed and undir.
Directed $\rightarrow \text{dens}(G) = \frac{E}{m(m-1)}$ Undirected $\rightarrow \text{dens}(G) = \frac{2E}{m(m-1)}$
The density is particularly important to identify the type of graph we are working with. A **Spars graph** has size $(E) : O(n)$, a **Dense graph** has $O(n^2)$. we use different data structures depending on the type.

Reciprocity of a graph $\rightarrow$ this is only applicable to digraphs and it measures the probability of seeing (u, v) given $(v, u) \rightarrow \text{rec}(G) = \frac{E_{\leftrightarrow}}{E}$ where $E_{\leftrightarrow}$ is the number of reciprocal links.

Operating on graphs $\rightarrow$ Since we defined graph with sets, we can also use set operations on them.

Complement $\rightarrow G^* = \{(u, v) \in V(G)^2 \mid (u, v) \notin E(G)\}$
Union $\rightarrow G \cup G' = E(G) \cup E(G')$ **Intersection** $\rightarrow G \cap G' = E(G) \cap E(G')$
Difference $\rightarrow G \setminus G' = E(G) - E(G')$

Line graph $\rightarrow$ the line graph of G is another graph where the edges of G become the nodes of $L(G)$ and the new nodes are connected if they are adjacent in the first graph.


Edge attributed graphs $\rightarrow$ Edges made graphs can be used for modelling relations so we also need the ability to attach attributes to them. In order to do that we define a new set of functions called **attribute functions** $\{f_1, f_2, \dots\}$ such that $f_i : E \rightarrow S_i$ where S_i is a generic set of values. These are the building blocks for attaching weights to edges. [unweighted] If we use each feature vector as the i -th row of a matrix, we create the **edge feature matrix** $E \in \mathbb{R}^{n \times d}$

Node attributed graph $\rightarrow$ what we just said for edges also applies to nodes, the resulting matrix is called **node feature matrix** $U \in \mathbb{R}^{n \times d}$

Representing graphs $\rightarrow$ there are multiple viable ways to represent a graph and they mostly depend on the use case.

Adjacency matrix $\rightarrow A \in \mathbb{R}^{n \times n}$ where $A_{u,v} = 1$ if $(u, v) \in E(G)$, it is always symmetric for undirected graphs ($A = A^T$). Very good for dense graphs, not so much for sparse graphs.

Coordinate list (COO) $\rightarrow$ Only stores the set of vertices 

Compressed sparse row (CSR) $\rightarrow$ it is a little trickier than the previous two, it has a 3-column structure that is composed as follows: ① Rows $\in \mathbb{R}^n \rightarrow$ index for node at j indicating where the set of nodes that are incident to j starts.
② Column $\in \mathbb{R}^E \rightarrow$ another set of links. ③ Value $\in \mathbb{R}^E \rightarrow$ attribute for / weight for each link

 Row: $\begin{bmatrix} 0 & 1 & 2 & 3 & 4 \end{bmatrix}$ Value is omitted for the example
Col: $\begin{bmatrix} 1 & 2 & 4 & 4 & 4 \end{bmatrix}$

Adjacency list $\rightarrow$ the entire idea is to use a list or set to store the nodes linked to a node u , using the graph above we have:
 $0 \rightarrow \{1, 2\}; 1 \rightarrow \{1\}; 2 \rightarrow \{1, 4\}; 3 \rightarrow \{1\}; 4 \rightarrow \{1\}$

Edge list $\rightarrow$ Simple list of tuples $\rightarrow [(0, 1, \text{value}), (0, 2), (1, 4) \dots]$
Incident matrix $\rightarrow$ rectangular matrix $\mathbb{R}^{n \times E}$ where the values are as follows:
Undirected $\rightarrow \{(u, v) \in E(G) \Leftrightarrow A_I(u, u, v) = 1 \wedge A_I(v, u, u) = 1, \text{ else } 0$
Directed $\rightarrow (u, v) \in E(G) \Leftrightarrow A_I(u, u, v) = 1 \wedge A_I(v, u, u) = -1, \text{ else } 0$
Example with the graph above (directed)

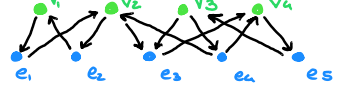
$\begin{bmatrix} (0, 1), (0, 2), (1, 4), (2, 4), (3, 1) \end{bmatrix} \rightarrow \begin{matrix} & e_1 & e_2 & e_3 & e_4 & e_5 \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 \end{bmatrix} \end{matrix}$

Hypergraphs $\rightarrow$ So far we have seen graphs where edges could only connect two nodes, if we remove this rule we have $e = \{V_1 \dots V_k\}$ with $k \geq 2$, also called a **hyperedge**. These new components allow us to model more complex relationships
Directed Hypergraph $\rightarrow H = (V, E), E: 2^V \rightarrow 2^V$ (powerset of $V \rightarrow$ set of all possible subsets of V). $e \in E$ has the form (U, W) where U is called **tail** and W is called **head**.

Undirected Hypergraph $\rightarrow$ Slightly simpler because $e \in E$ has form $\{U, W\}, U \cup W$ therefore $E \subseteq 2^V$

Order, Size and Density for hypergraphs $\rightarrow$ order is still $n = |V|$ and size is still $E = |E|$; the density is calculated as follows: $\text{dens}(G) = \frac{E}{2^{\binom{n-1}{2}}}$; $\text{dens}(G) = \frac{E}{2^n}$

Representing a hypergraph with an incidence structure $\rightarrow$ the idea is to create a structure where both the nodes and links of the original graph are nodes of the structure, let's see a quick example.

 $V = \{V_1, V_2, V_3, V_4\}$ $E = \{e_1, e_2, e_3, e_4, e_5\}$
 $e_1 = \{\{V_1\}, \{V_2\}\}$ $e_4 = \{\{V_3\}, \{V_4, V_6\}\}$
 $e_2 = \{\{V_2\}, \{V_4\}\}$ $e_5 = \{\{V_4\}, \{V_3\}\}$
 $e_3 = \{\{V_2, V_3\}, \{V_4\}\}$

this can be easily translated into a matrix similar to the one seen for classical graphs.
graph: $\begin{matrix} & e_1 & e_2 & e_3 & e_4 & e_5 \\ \begin{matrix} V_1 \\ V_2 \\ V_3 \\ V_4 \end{matrix} & \begin{bmatrix} 1 & 1 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 1 \end{bmatrix} \end{matrix}$
 1° component, 1 if tail
 2° component, 1 if head

the membership of vertices in sets or in hypergraphs can be represented by using a **Bipartite graph** or **two-mode network**.

Bipartite graph $\rightarrow$ a network where we have two types of nodes and edges can only connect nodes of **different types**, obviously a node can only have one type.

the incidence matrix for a bipartite graph is $B_I \in \mathbb{R}^{N_1 \times N_2}$ and the values are $B_{u,v} = \begin{cases} 1 & \text{if } u \in C \\ 0 & \text{otherwise} \end{cases}$


Bipartite structures are not always great for ML, we'd like to map such stuff into a "normal" graphs. We can do that with approaches such as **one-mode projection**

Interacting networks $\rightarrow$ most real systems are way more complex than normal graphs, therefore we need structures to represent such systems.

General multi-layer network $\rightarrow$ defined by a tuple $M = (Y, \vec{G}, \vec{B})$ where:

1 $\rightarrow Y = \{\alpha \mid \alpha \in \{1 \dots H\}\}$ is the set of layers
2 $\rightarrow \vec{G} = \{G_1 \dots G_H\} : G_\alpha = (V_\alpha, E_\alpha)$ is the set of graphs where the links inside a graph are called **intralinks**.

3 $\rightarrow \vec{B} \in M_n \times M_n$ is the set of bipartite graphs with $B_{i,j} = \{V_A, V_B, E_{i,j}\}$

Multilayer network $\rightarrow$ multi-layer structure that maps 1 to 1 nodes between layers (reflex nodes), intralinks can only connect reflex nodes.

A multilayer network can be both **with** and **without** intralinks. $Y \mapsto Y_{ij} = \{(1, 2)\}$

With $\rightarrow$ used when reflex nodes have a distinct identity 
Without $\rightarrow$ used when there is no distinction between reflexes $\rightarrow M = (Y, \vec{G}, \vec{B})$

For multilayer networks **with** intralinks, we can represent them using a single **Synja-adjacency matrix** of the following form.

$A \in \mathbb{R}^{n \times n}$ where $n = |V|$ and H is the number of layers. The matrix is made of blocks that define the intralinks and interlinks.
 $A_i = \begin{bmatrix} I & A_{i,1} \\ A_{1,i} & I \end{bmatrix}$ the identity matrices $I \in \mathbb{R}^{n \times n}$ define the intralinks between the layers; the $A_{1 \dots H}$ matrices are instead used for interlinks
 $A_{i,j} = \begin{cases} A_{i,j} & \text{if } i \neq j \\ \delta I_{i,j} & \text{if } i = j \end{cases}$

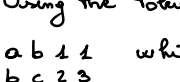
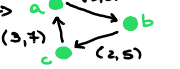
For multilayer networks **without** intralinks, we can encode them using one adjacency matrix for each layer (H total matrices). We can also represent them using a multi (di)graph $\rightarrow$  the color is used to distinguish the type of relationship.

Multi-slice networks $\rightarrow$ a more specific type where the intralinks can only exist in a temporal succession. $E_{i,i+1} = \{(i, \alpha), (i, \alpha+1) \mid i \in \{1 \dots n\}\}$

 the synja adjacency matrix is very similar to the one above but the identity matrices are only on the blocks i above the diagonal.

Network of networks $\rightarrow$ a multilayer network with a **supernetwork**, basically the supernetwork decides which layers have their reflex nodes linked. It is used to model real world constraints over the data.

Time in networks $\rightarrow$ in our case, time allows us to model real world processes that are dynamic in nature. We can represent the evolution over time with a tubular form, a small example is:

Using the tubular form (from, to, start, duration) we define the table below
 which, in graph form, is $\rightarrow$  also called **interval graph**

the example above is also called **link lifespan** and assumes that the nodes are active through the entire lifetime of the graph.
We can also add a table for node lifetimes if needed.

For more specific use cases, where the data we have comes from snapshots, we can use **time-varying graphs**, to which we can apply either coarse or variable resolution to capture the graph's evolution over time.